ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID



Introducción a Machine Learning con Python

Luis Garmendia Salvador

Departamento de Ingeniería del Software e Inteligencia Artificial
Facultad de Informática, UCM

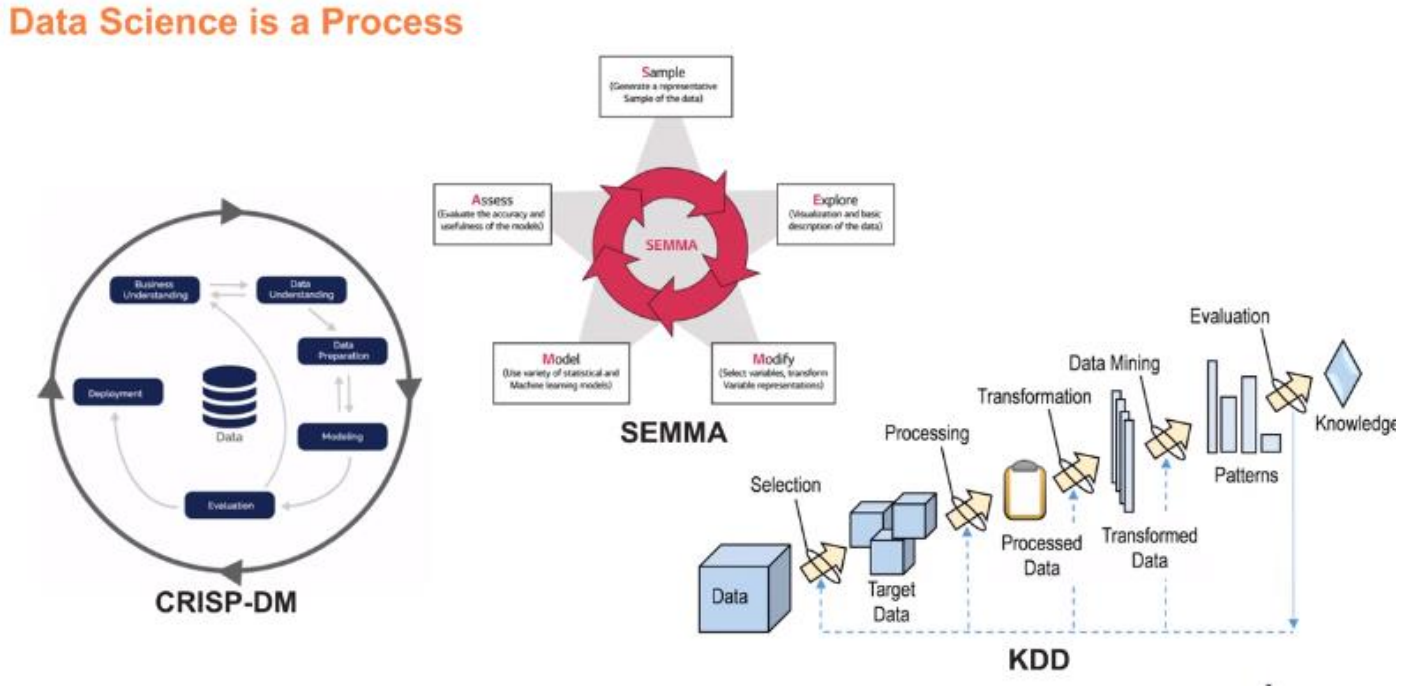
1

Introducción a Machine Learning con Python

- Introducción a las ciencias de datos
- Aprendizaje supervisado y no supervisado
- Instalación de sklearn (pip install sklearn)
- Datos de Iris
- Pasos de un clasificador de Machine Learning
- Ejemplo de clasificador de Naive Bayes
- Ejemplo de Clustering (GMM)
- Reducción de la dimensionalidad (PCA)
- Matriz de Confusión
- Validación de un Modelo (bias-varianza)
- Vectorización

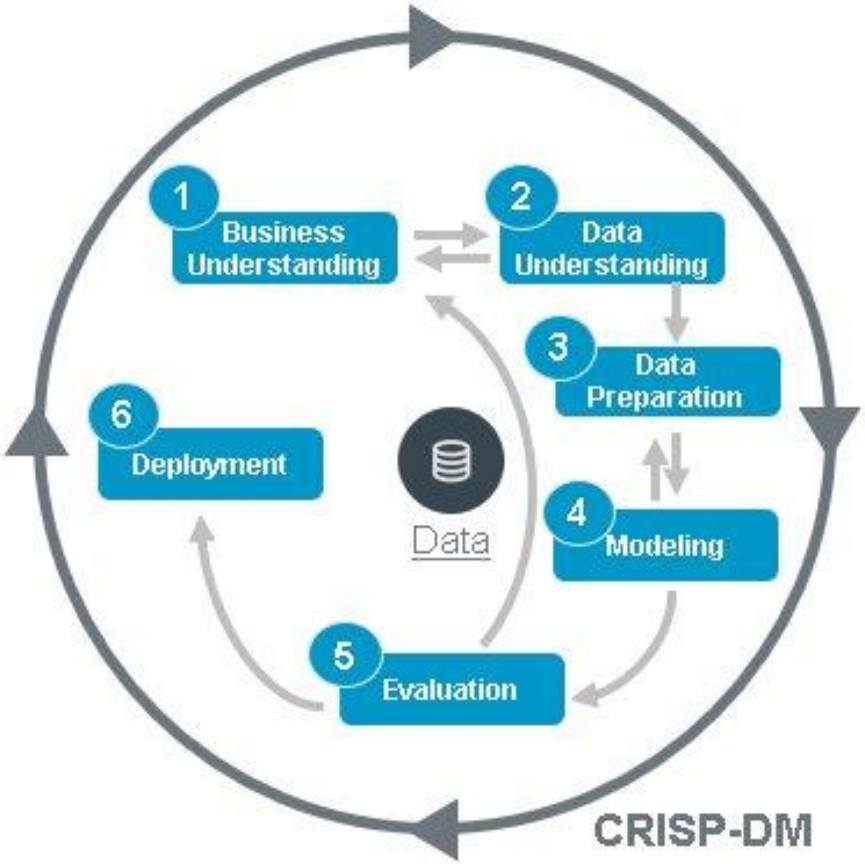


Las ciencias de datos son un proceso

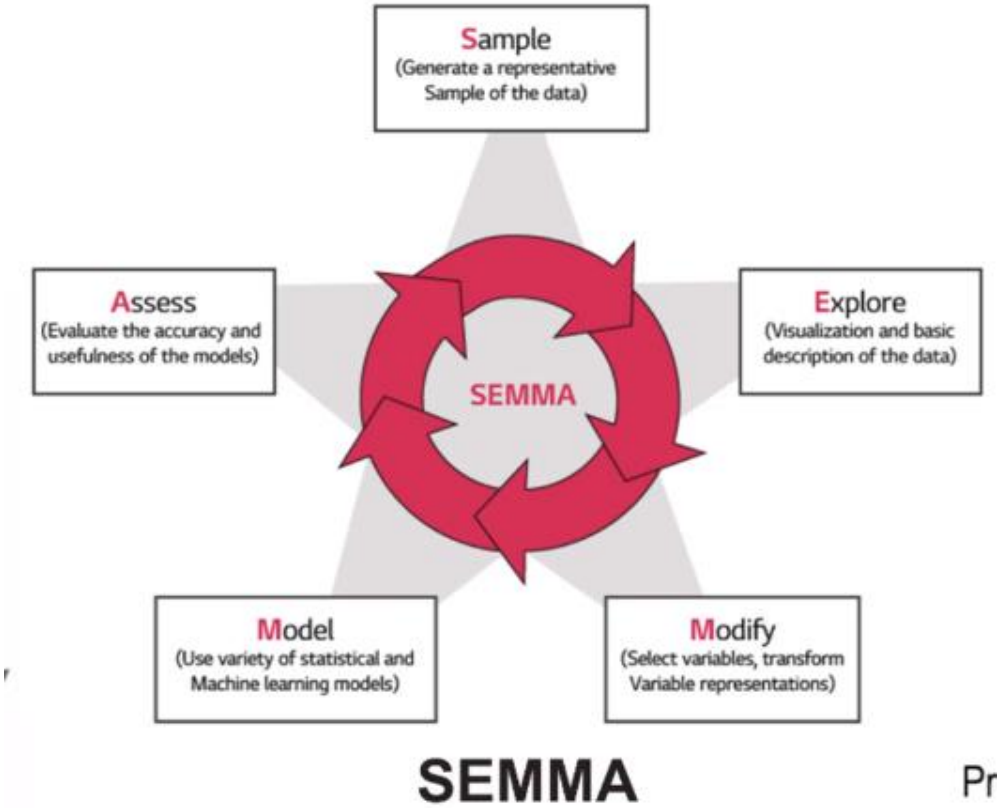


Fases según el documento CRISP-DM (70 páginas)

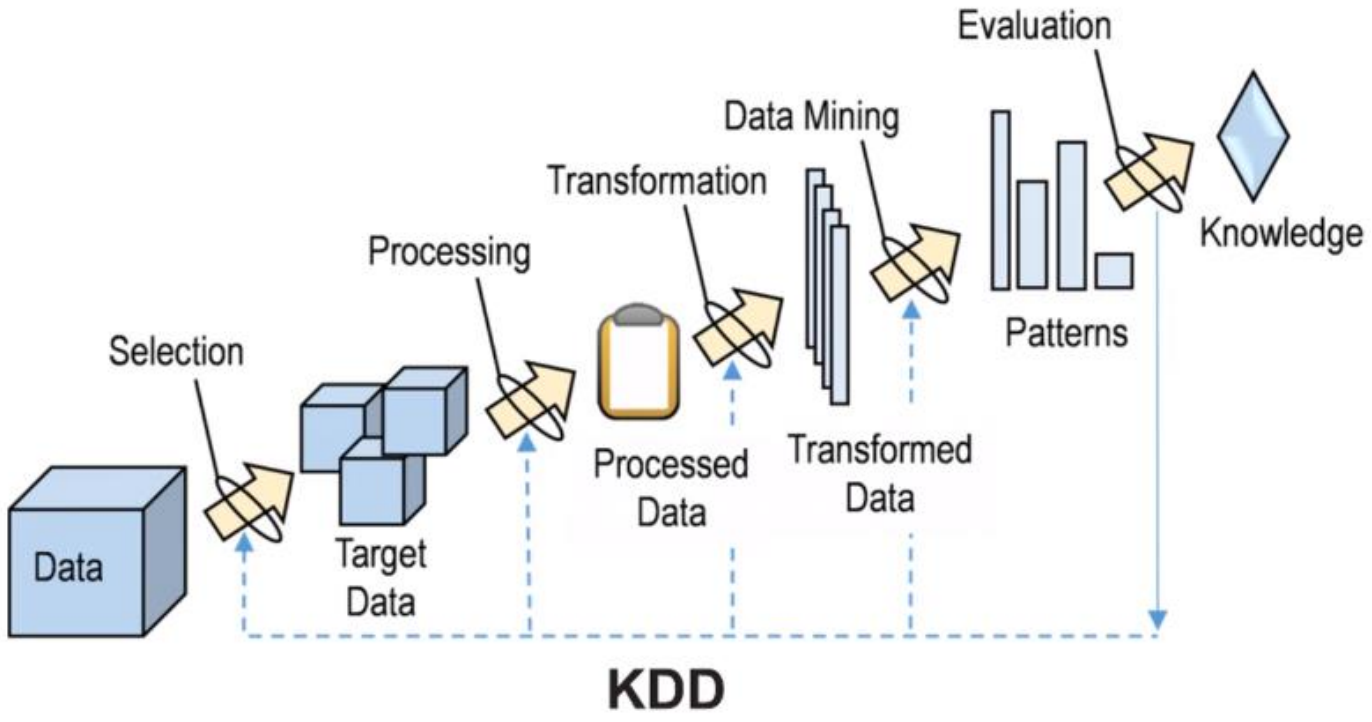
1- Bussiness Understanding
2- Data Understanding
3- Data preparation
4 – Modeling
5 – Evaluation
6 - Deployment



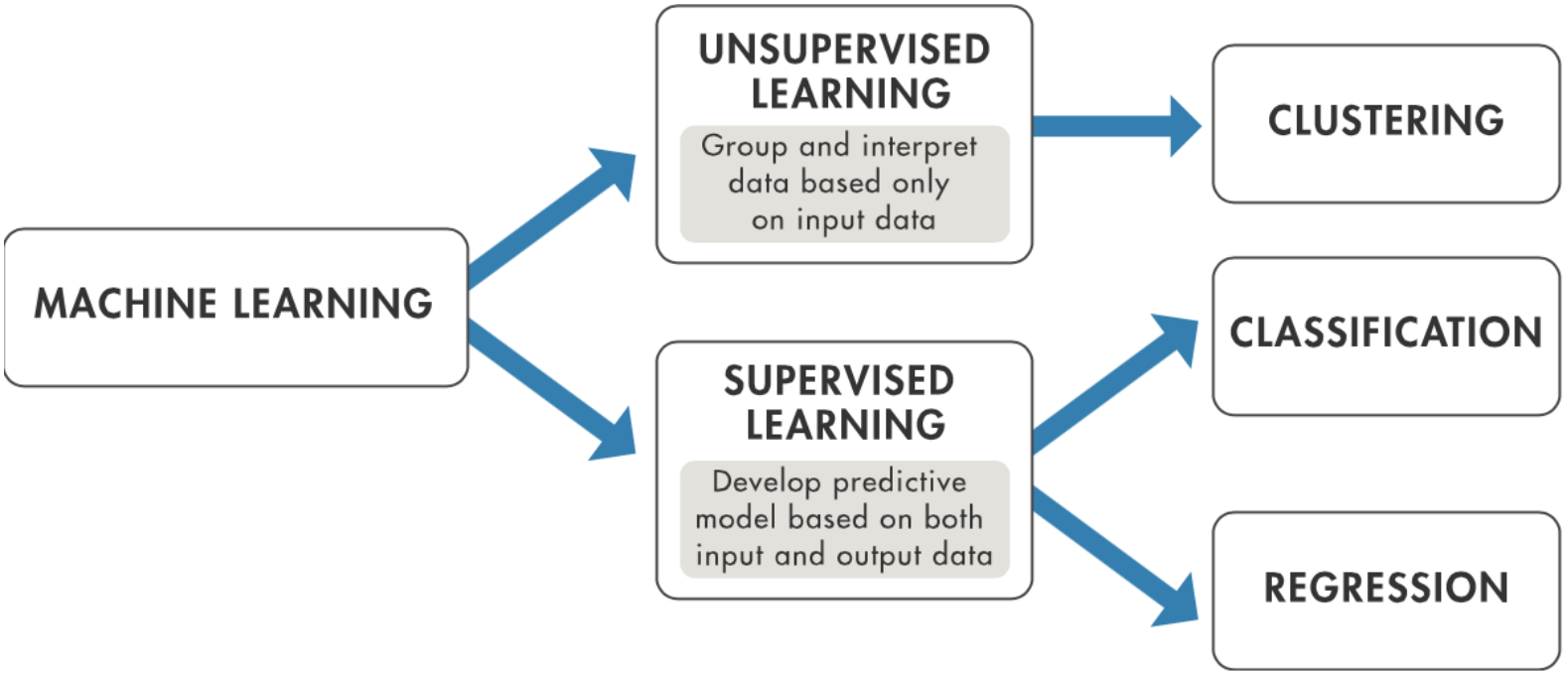
SEMMA


Sample
(Generate a representative Sample of the data)
Explore
(Visualization and basic description of the data)
Modify
(Select variables, transform Variable representations)
Model
(Use variety of statistical and Machine learning models)
Assess
(Evaluate the accuracy and usefulness of the models)
SEMMA
SEMMA
Pr

KDD


Evaluation
Data Mining
Transformation
Processing
Selection
Knowledge
Patterns
Transformed Data
Processed Data
Target Data
Data
KDD

4 - Modelado


MACHINE LEARNING
UNSUPERVISED LEARNING
Group and interpret data based only on input data
SUPERVISED LEARNING
Develop predictive model based on both input and output data
CLUSTERING
CLASSIFICATION
REGRESSION

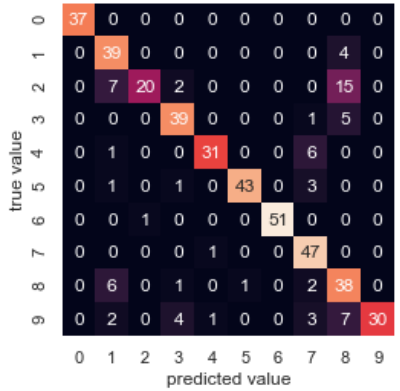
5 - Evaluación



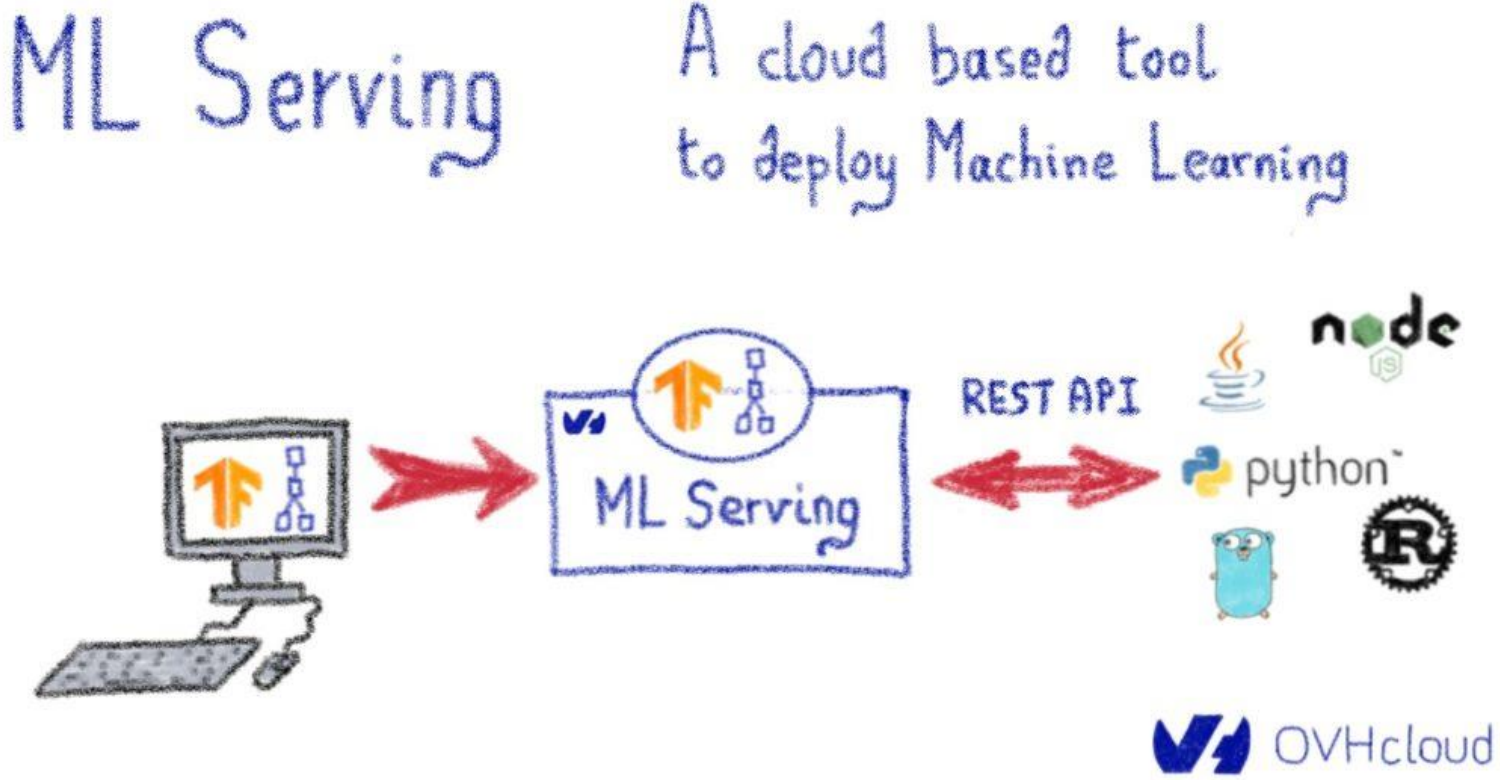

```
In [83]: from sklearn.metrics import confusion_matrix

mat = confusion_matrix(ytest, y_model)

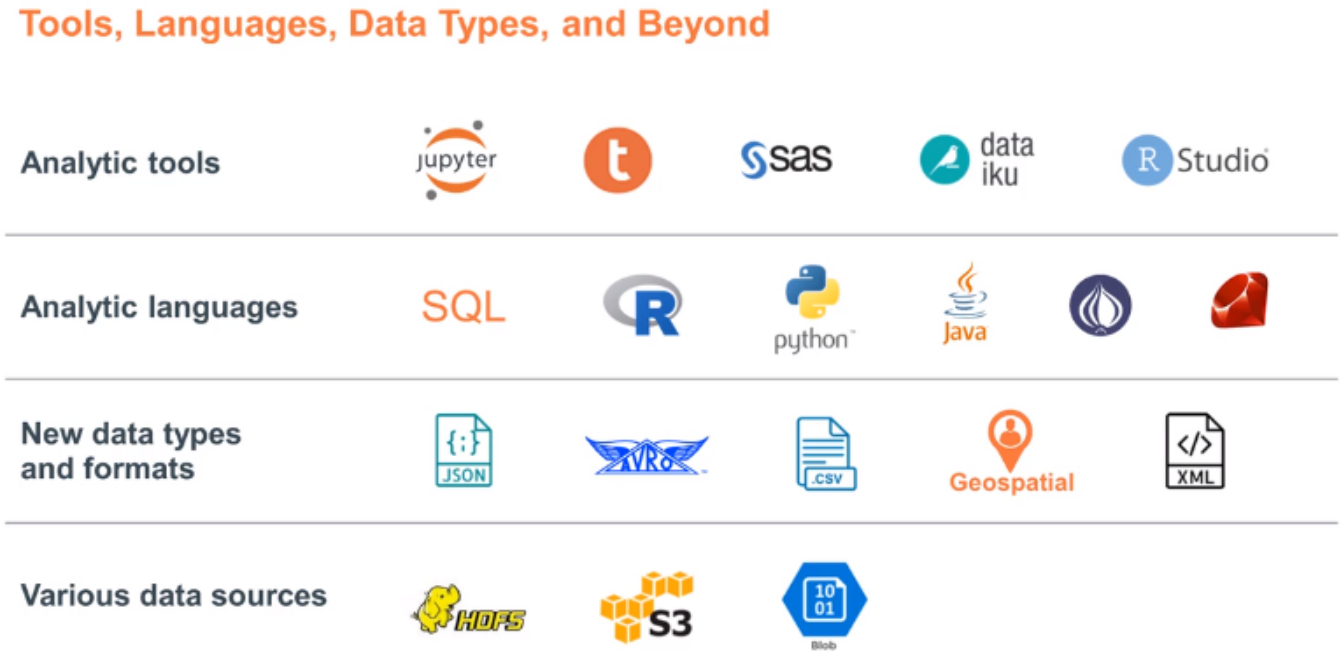
sns.heatmap(mat, square=True, annot=True, cbar=False)
plt.xlabel('predicted value')
plt.ylabel('true value');
```



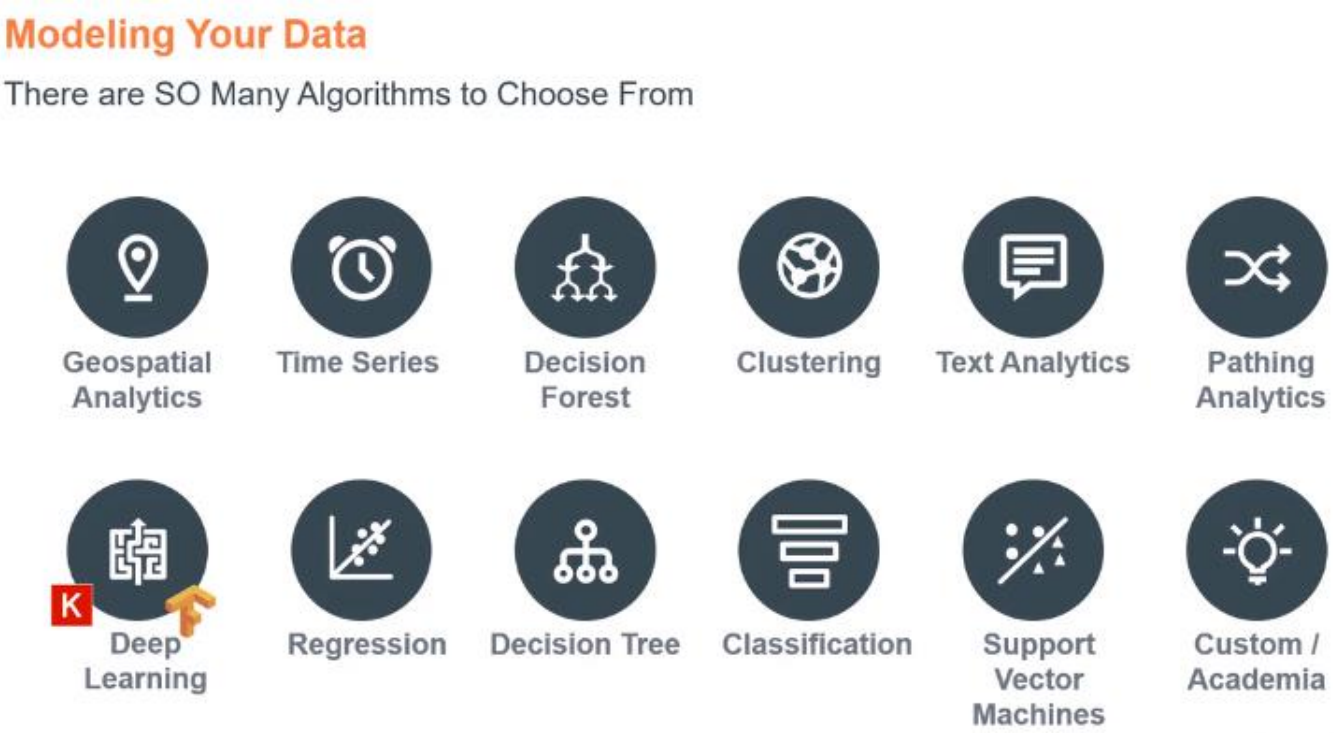
6 - Desplegado



Herramientas de Ciencias de Datos

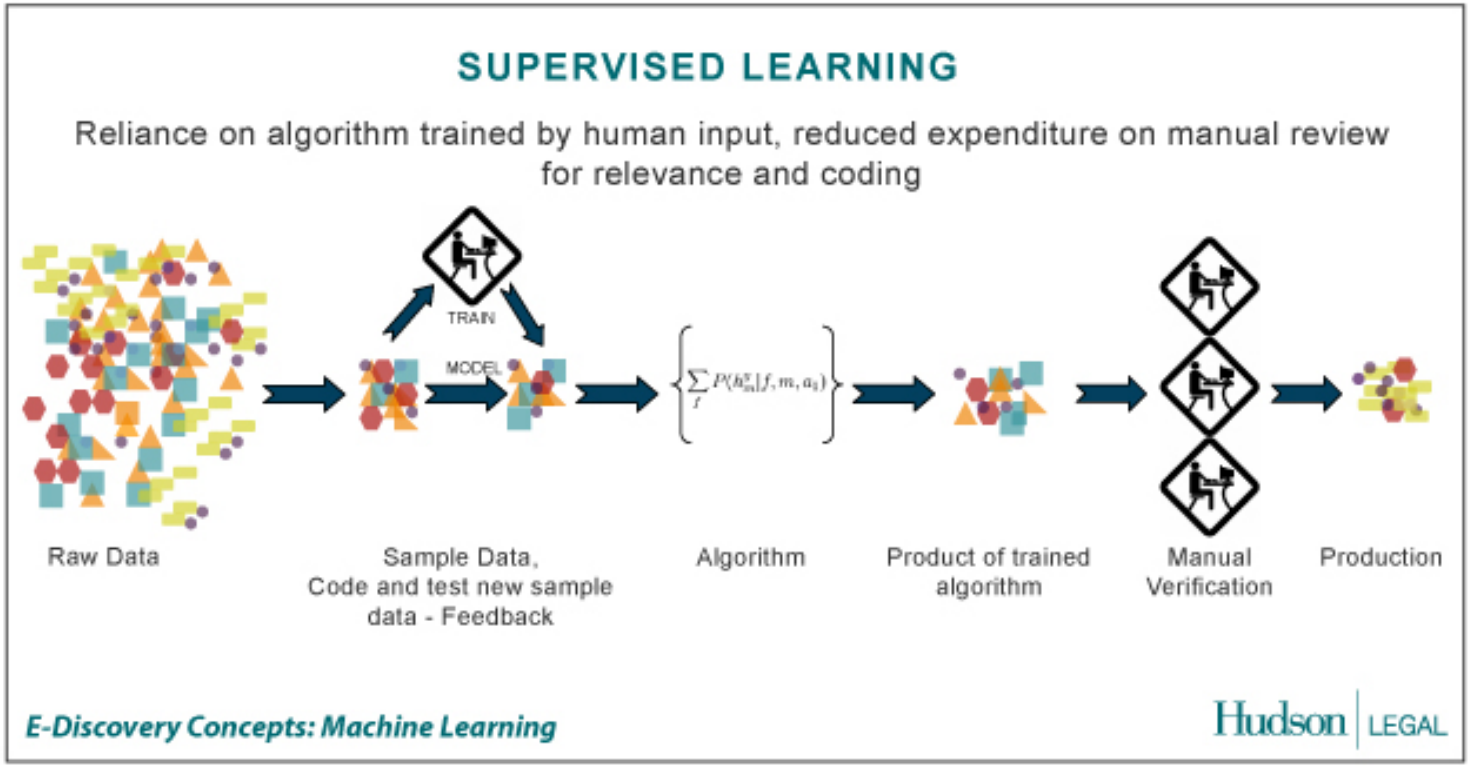


Algoritmos de Ciencias de Datos



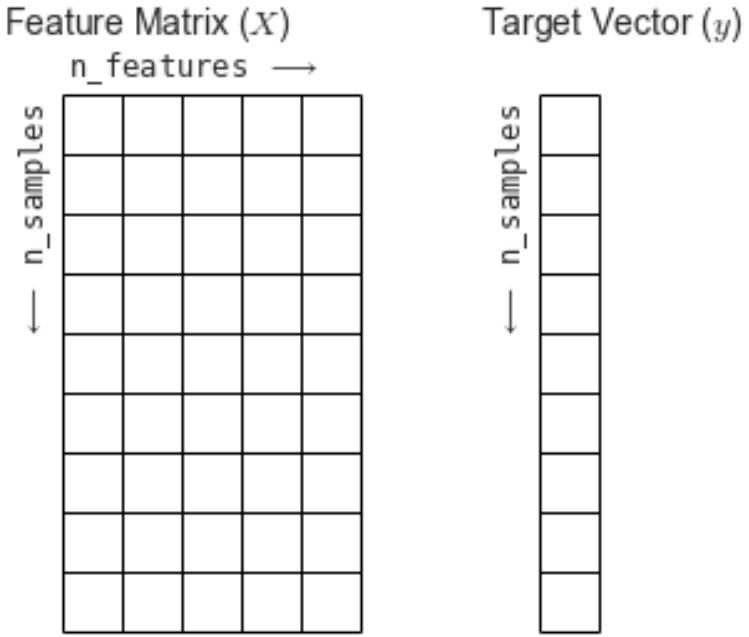
Aprendizaje Supervisado

Se conocen las clases. Los datos se etiquetan. Ejemplo: los correos son spam o legítimos



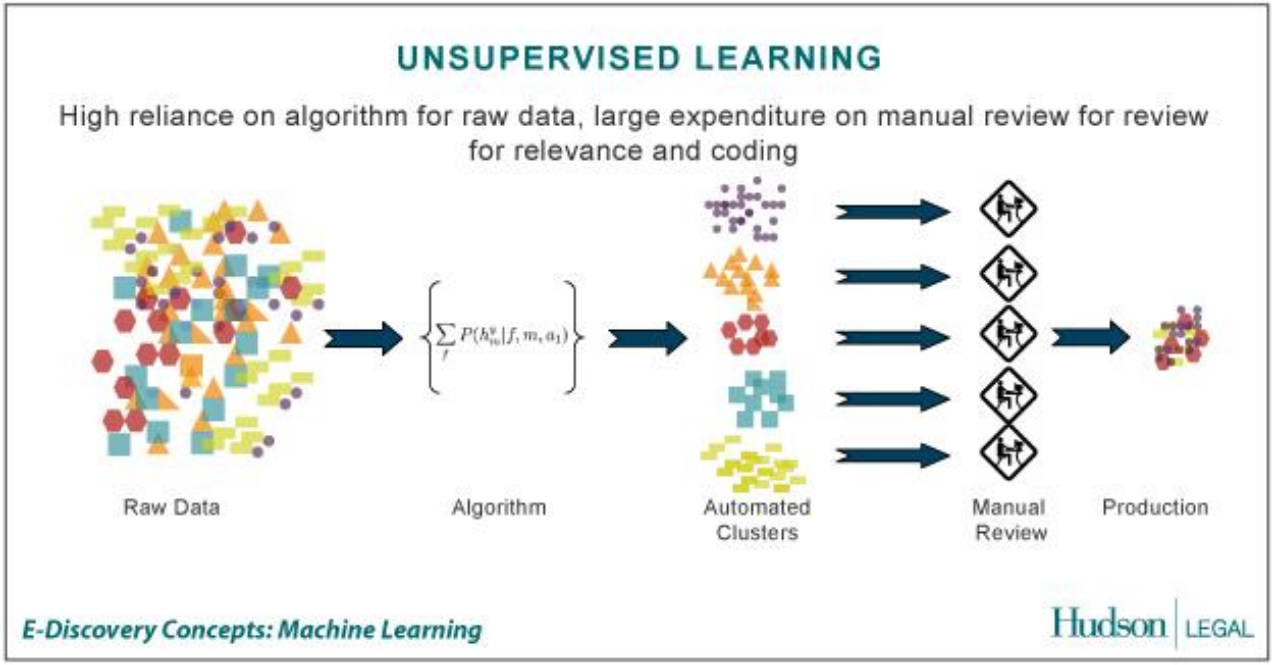
Objetivo: Target array

Los datos de una matriz X se contienen en un array de NumPy o en una Serie de Panda
La columna objetivo Y es la variable dependiente que queremos predecir a partir de X..

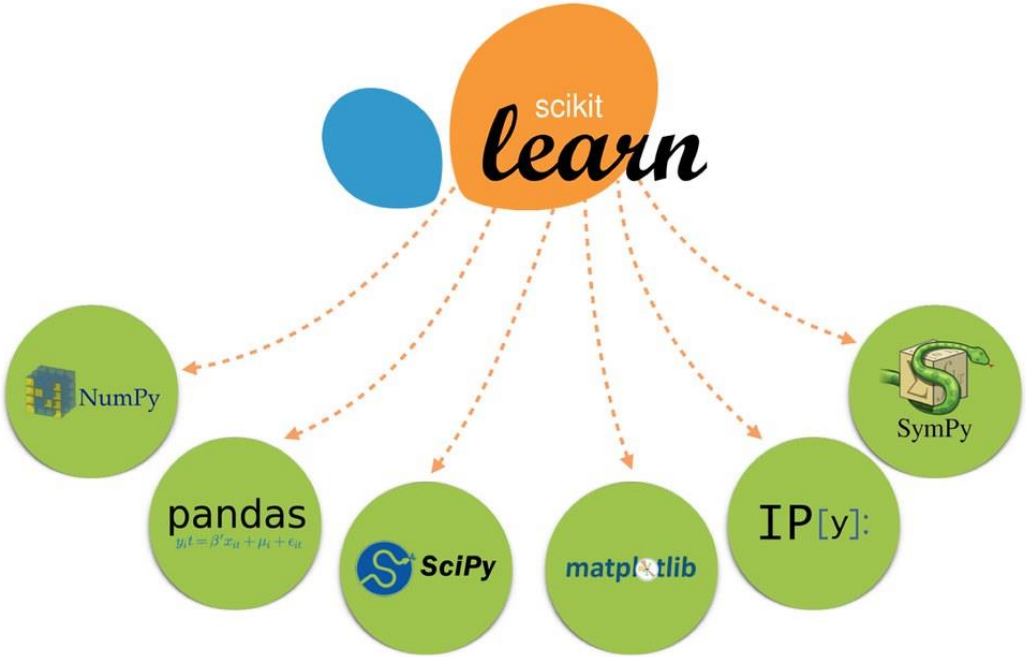


Aprendizaje No Supervisado

No se conocen las clases. Los datos no se etiquetan. No se conoce estructura o forma de organizar los datos.



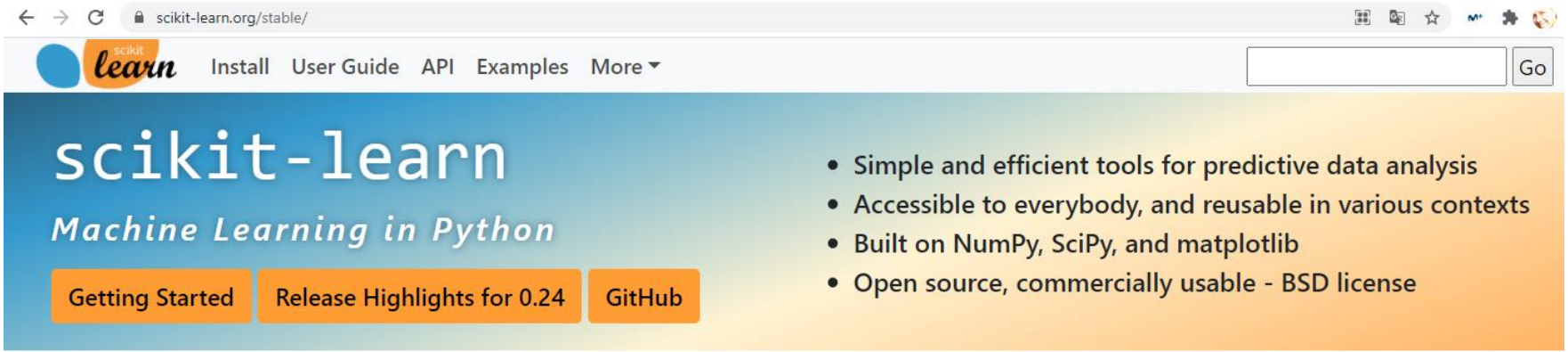
Scikit-learn



Luís Garmendia

Scikit-learn

Librería de Machine Learning en Python



https://scikit-learn.org/stable/

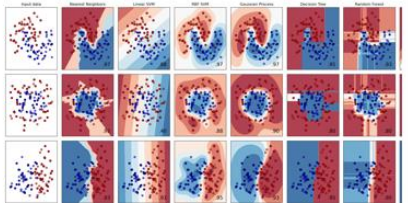
Scikit-learn

https://scikit-learn.org/stable/

Classification

Identifying which category an object belongs to.

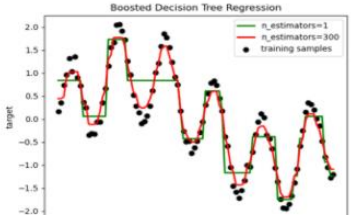
Applications: Spam detection, image recognition.
Algorithms: SVM, nearest neighbors, random forest, and more...



Regression

Predicting a continuous-valued attribute associated with an object.

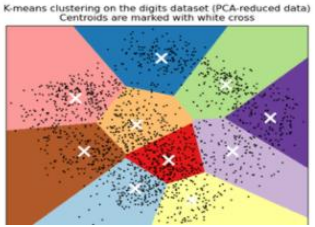
Applications: Drug response, Stock prices.
Algorithms: SVR, nearest neighbors, random forest, and more...



Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, spectral clustering, mean-shift, and more...



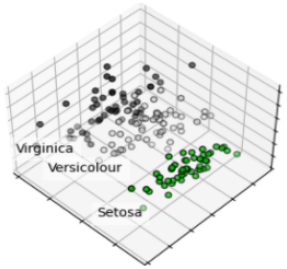
Scikit-learn

https://scikit-learn.org/stable/

Dimensionality reduction

Reducing the number of random variables to consider.

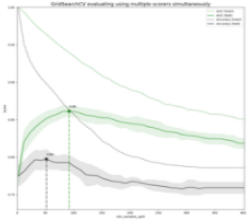
Applications: Visualization, Increased efficiency
Algorithms: k-Means, feature selection, non-negative matrix factorization, and more...



Model selection

Comparing, validating and choosing parameters and models.

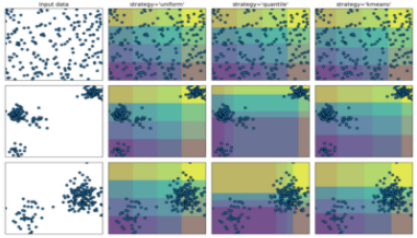
Applications: Improved accuracy via parameter tuning
Algorithms: grid search, cross validation, metrics, and more...



Preprocessing

Feature extraction and normalization.

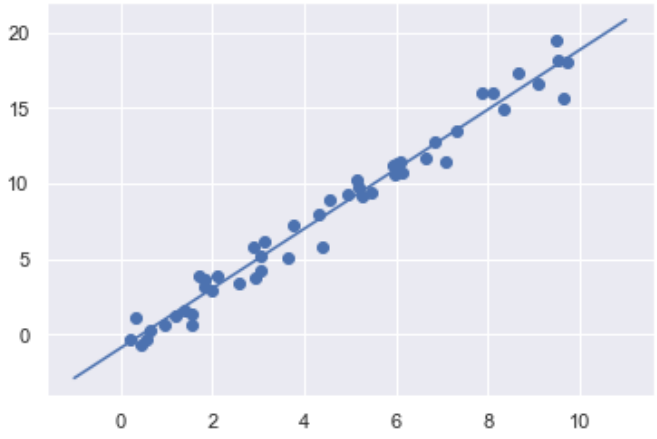
Applications: Transforming input data such as text for use with machine learning algorithms.
Algorithms: preprocessing, feature extraction, and more...



Pasos en Machine learning

1. Elegir un Modelo
2. Elegir Parámetros
3. Elegir matriz X y objetivo Y
4. Ajustar el modelo: **fit()**
5. Aplicar el modelo
    1. Para aprendizaje supervidado: **predict()**.
    2. Para aprendizaje no supervisado: método **transform() o predict()**.

Regresion Lineal (supervised)



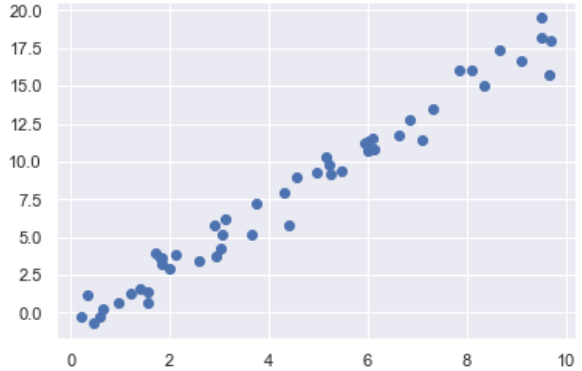
Luís Garmendia

Linear Regression (supervised)

```
import matplotlib.pyplot as plt
import numpy as np
rng = np.random.RandomState(42)
x = 10 * rng.rand(50)
y = 2 * x - 1 + rng.randn(50)
plt.scatter(x, y);
```

In [20]:
```
import matplotlib.pyplot as plt
import numpy as np

rng = np.random.RandomState(42)
x = 10 * rng.rand(50)
y = 2 * x - 1 + rng.randn(50)
plt.scatter(x, y);
```



In [21]:
```
x
```

Out[21]:
```
array([3.74540119, 9.50714306, 7.31993942, 5.98658484, 1.5601864 ,
       1.5599452 , 0.58083612, 8.66176146, 6.01115012, 7.08072578,
```

Install sklearn
pip install sklearn

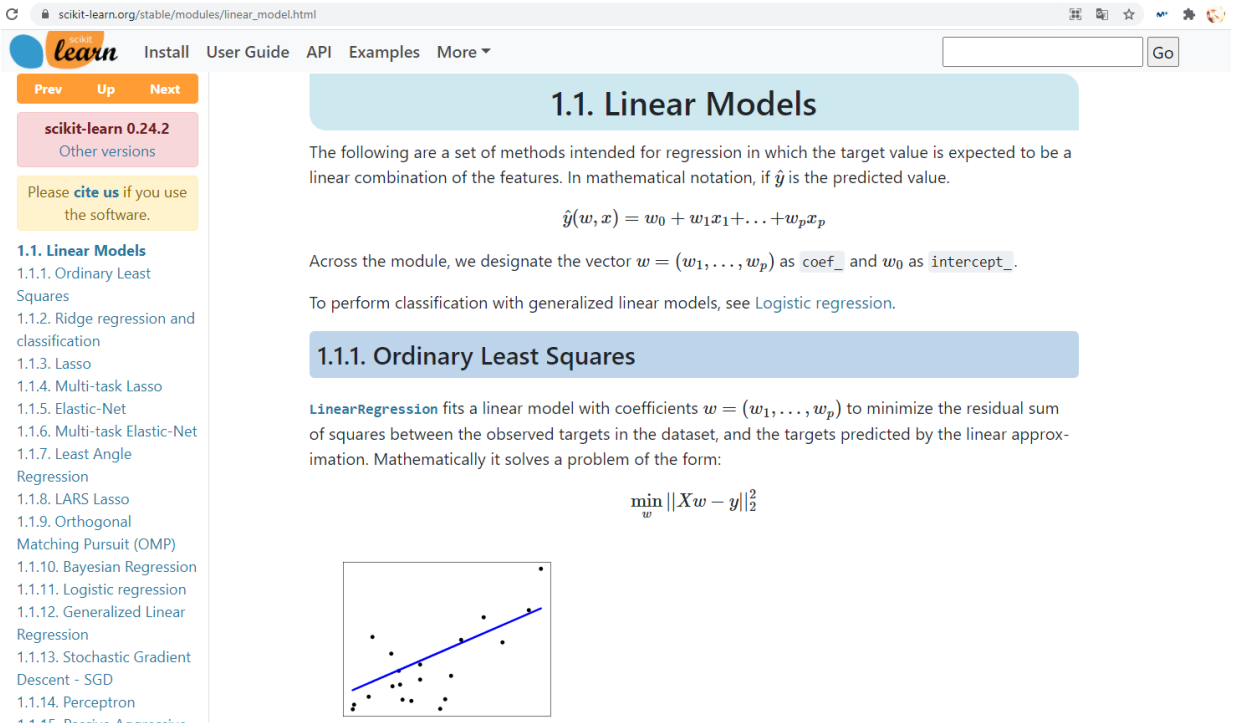
In [23]:
```
pip install sklearn
```

```
Collecting sklearn
  Downloading sklearn-0.0.tar.gz (1.1 kB)
Requirement already satisfied: scikit-learn in c:\users\luis\anaconda3\lib\site-packages (from sklearn) (0.22.1)
Requirement already satisfied: scipy>=0.17.0 in c:\users\luis\anaconda3\lib\site-packages (from scikit-learn->sklearn) (1.4.1)
Requirement already satisfied: numpy>=1.11.0 in c:\users\luis\anaconda3\lib\site-packages (from scikit-learn->sklearn) (1.18.1)
Requirement already satisfied: joblib>=0.11 in c:\users\luis\anaconda3\lib\site-packages (from scikit-learn->sklearn) (0.14.1)
Building wheels for collected packages: sklearn
  Building wheel for sklearn (setup.py): started
  Building wheel for sklearn (setup.py): finished with status 'done'
  Created wheel for sklearn: filename=sklearn-0.0-py2.py3-none-any.whl size=1320 sha256=264abce7084e9dc81b0a059edb1b2ac99385b71
d0ca342a6e5b642fed1b82dc7
  Stored in directory: c:\users\luis\appdata\local\pip\cache\wheels\46\ef\c3\157e41f5ee1372d1be90b09f74f82b10e391eaacca8f22d33e
Successfully built sklearn
Installing collected packages: sklearn
Successfully installed sklearn-0.0
Note: you may need to restart the kernel to use updated packages.
```

In [24]:
```
from sklearn.linear_model import LinearRegression
```

Documentación

https://scikit-learn.org/stable/modules/linear_model.html



1 – Instanciar el Modelo

```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)
```

```
In [24]: from sklearn.linear_model import LinearRegression

In [25]: model = LinearRegression(fit_intercept=True)
         model

Out[25]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

2 - X, y

X = x[:, np.newaxis]
X.Shape

```
In [28]: x

Out[28]: array([3.74540119, 9.50714306, 7.31993942, 5.98658484, 1.5601864 ,
                1.5599452 , 0.58083612, 8.66176146, 6.01115012, 7.08072578,
                0.20584494, 9.69909852, 8.32442641, 2.12339111, 1.81824967,
                1.8340451 , 3.04242243, 5.24756432, 4.31945019, 2.9122914 ,
                6.11852895, 1.39493861, 2.92144649, 3.66361843, 4.56069984,
                7.85175961, 1.99673782, 5.14234438, 5.92414569, 0.46450413,
                6.07544852, 1.70524124, 0.65051593, 9.48885537, 9.65632033,
                8.08397348, 3.04613769, 0.97672114, 6.84233027, 4.40152494,
                1.22038235, 4.9517691 , 0.34388521, 9.09320402, 2.58779982,
                6.62522284, 3.11711076, 5.20068021, 5.46710279, 1.84854456])

In [29]: X = x[:, np.newaxis]
         X.shape

Out[29]: (50, 1)

In [30]: X

Out[30]: array([[3.74540119],
                [9.50714306],
                [7.31993942],
                [5.98658484],
                [1.5601864 ],
                [1.5599452 ],
                [0.58083612],
                [8.66176146],
```

3 – Ajustar el Modelo

model.fit(X, y)

model.coef_
model.intercept_

```
In [31]: model.fit(X, y)
Out[31]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)

In [32]: model.coef_
Out[32]: array([1.9776566])

In [33]: model.intercept_
Out[33]: -0.9033107255311164
```

4 - Predicción para datos nuevos

```
xfit = np.linspace(-1, 11)

Xfit = xfit[:, np.newaxis]

yfit = model.predict(Xfit)
```

```
In [41]: xfit = np.linspace(-1, 11)

In [42]: Xfit = xfit[:, np.newaxis]
         yfit = model.predict(Xfit)

In [43]: Xfit.shape

Out[43]: (50, 1)

In [44]: yfit.shape

Out[44]: (50,)

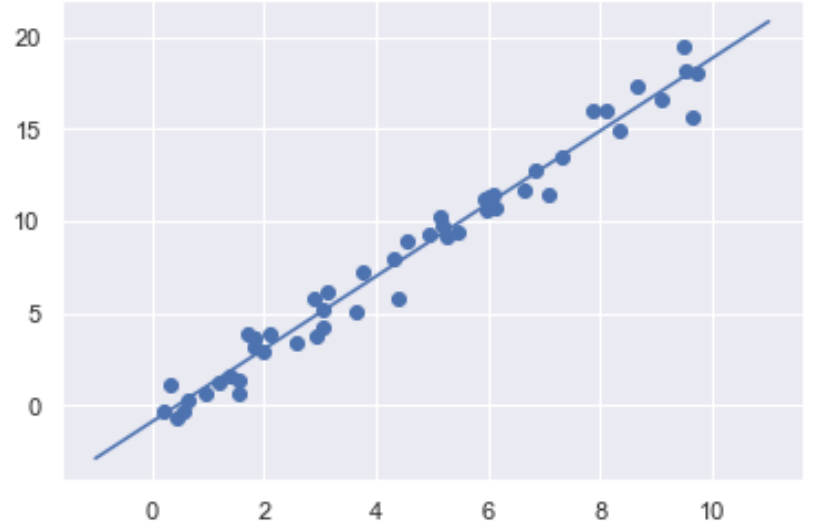
In [45]: Xfit

Out[45]: array([[-1.        ],
                [-0.75510204],
                [-0.51020408],
                [-0.26530612],
                [-0.02040816],
                [ 0.2244898 ],
```

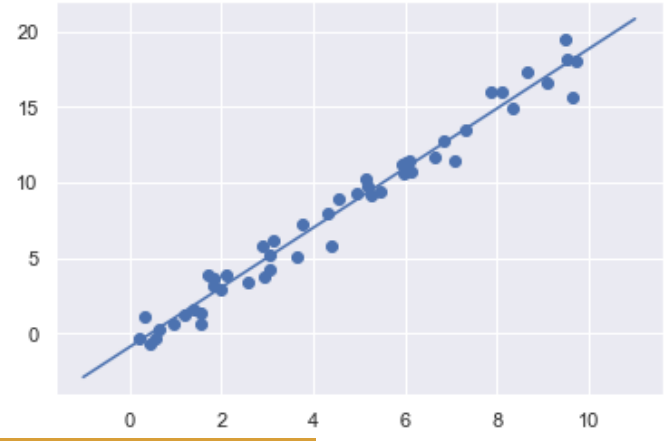
5 - Visualización de los datos

plt.scatter(x, y)
plt.plot(xfit, yfit);

```
In [46]: plt.scatter(x, y)
         plt.plot(xfit, yfit);
```



Clasificación con Naive Bayes (supervisado)

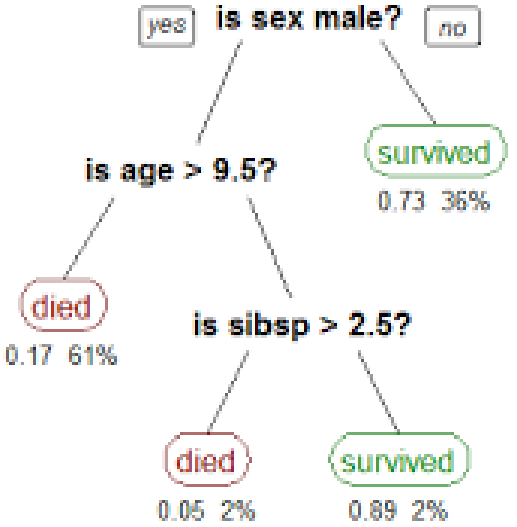


Luís Garmendia

Naïve Bayes

El modelo Naive Bayes es rápido y simple, válido para muchas dimensiones y sin parámetros.

$$P(L \mid \text{features}) = \frac{P(\text{features} \mid L)P(L)}{P(\text{features})}$$



Trees

Ejercicio: Datos de iris

```
import seaborn as sns
iris = sns.load_dataset('iris')
iris.head()
```

In [18]:
```
import seaborn as sns
iris = sns.load_dataset('iris')
iris.head()
```

Out[18]:

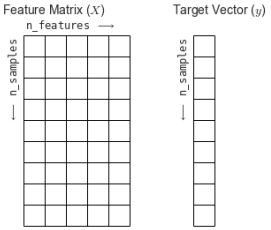
	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Preparación de Iris X e y

```
import seaborn as sns
iris = sns.load_dataset('iris')
iris.head()

X_iris = iris.drop('species', axis=1)
X_iris.shape

y_iris = iris['species']
y_iris.shape
```




```
In [51]: import seaborn as sns
iris = sns.load_dataset('iris')
iris.head()
```

Out[51]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [52]: X_iris = iris.drop('species', axis=1)
X_iris.shape
```

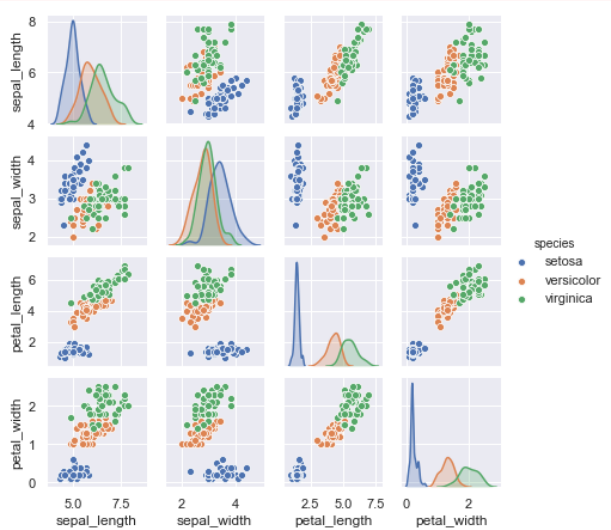
Out[52]: (150, 4)

```
In [53]: y_iris = iris['species']
y_iris.shape
```

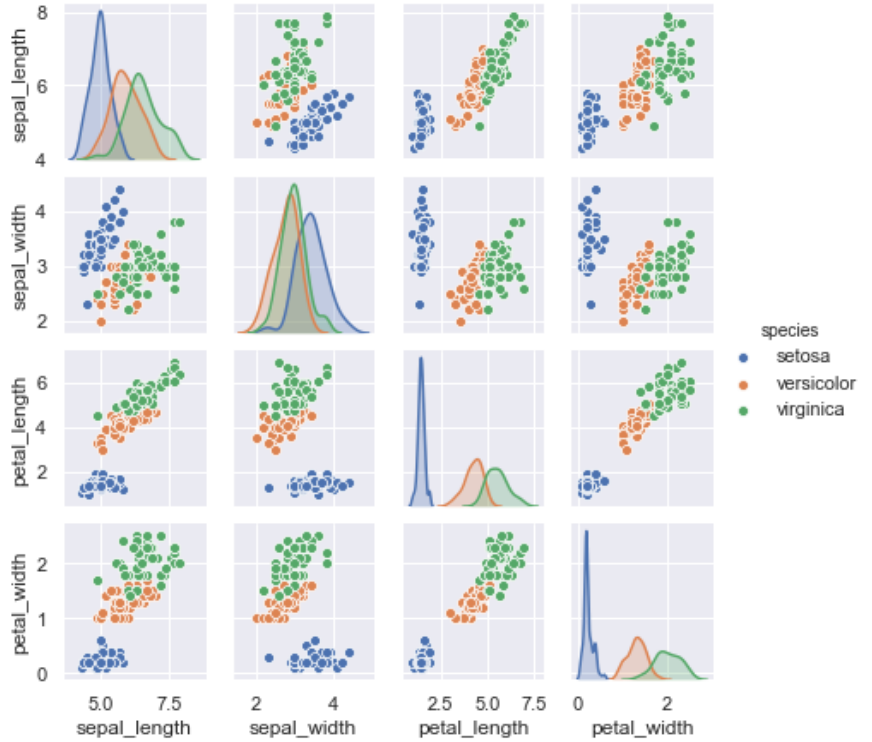
Out[53]: (150,)

Visualización de datos

```
%matplotlib inline
import seaborn as sns;
sns.set()
sns.pairplot(iris, hue='species', size=1.5);
```



Visualización de datos



Iris train_test_split

```
from sklearn.model_selection import train_test_split
Xtrain, Xtest, ytrain, ytest = train_test_split(X_iris, y_iris,
random_state=1)
```

```
In [54]: from sklearn.model_selection import train_test_split
         Xtrain, Xtest, ytrain, ytest = train_test_split(X_iris, y_iris,
                                                         random_state=1)

In [55]: Xtrain.shape

Out[55]: (112, 4)

In [56]: ytrain.shape

Out[56]: (112,)

In [57]: Xtest.shape

Out[57]: (38, 4)
```

Instanciación y ajuste (fit) del modelo

```
from sklearn.naive_bayes import GaussianNB

# 1. choose model class
model = GaussianNB()

# 2. instantiate model
model.fit(Xtrain, ytrain)

# 3. fit model to data
y_model = model.predict(Xtest)
```

Precisión: Accuracy_score

from sklearn.metrics import accuracy_score
accuracy_score(ytest, y_model)

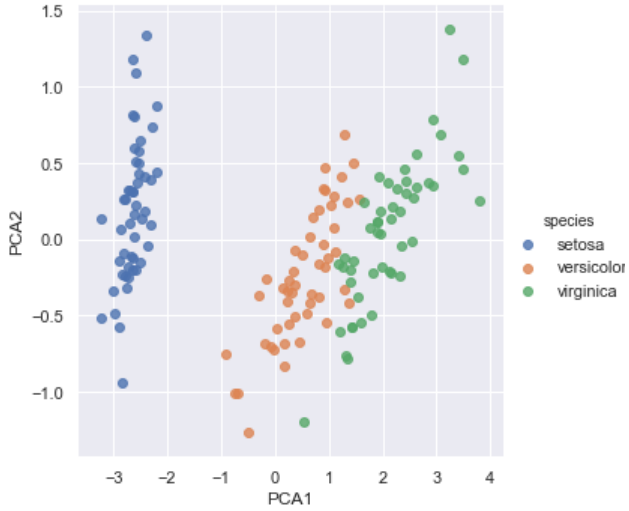
```
In [59]: from sklearn.naive_bayes import GaussianNB # 1. choose model class
         model = GaussianNB()                       # 2. instantiate model
         model.fit(Xtrain, ytrain)                  # 3. fit model to data
         y_model = model.predict(Xtest)

In [60]: from sklearn.metrics import accuracy_score
         accuracy_score(ytest, y_model)

Out[60]: 0.9736842105263158
```

El clasificador de Naïve Bayes es efectivo en un 97% para este dataset.

Unsupervised example: Iris dimensionality



Luís Garmendia

PCA: Principal component analysis

```
from sklearn.decomposition import PCA # 1. Choose the model class
model = PCA(n_components=2)           # 2. Instantiate the model with
hyperparameters
model.fit(X_iris)                                          # 3. Fit to data. Notice
y is not specified!
X_2D = model.transform(X_iris)        # 4. Transform the data to two dimensions
```

```
In [61]: from sklearn.decomposition import PCA  # 1. Choose the model class
         model = PCA(n_components=2)            # 2. Instantiate the model with hyperparameters
         model.fit(X_iris)                      # 3. Fit to data. Notice y is not specified!
         X_2D = model.transform(X_iris)         # 4. Transform the data to two dimensions

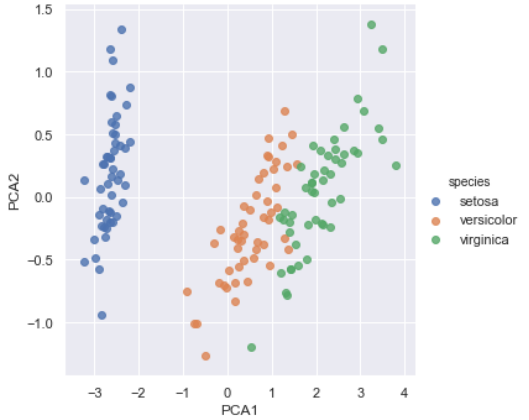
In [62]: X_2D.shape

Out[62]: (150, 2)
```

PCA: Principal component

```
iris['PCA1'] = X_2D[:, 0]
iris['PCA2'] = X_2D[:, 1]
sns.lmplot(x="PCA1", y= "PCA2", hue='species', data=iris, fit_reg=False);
```

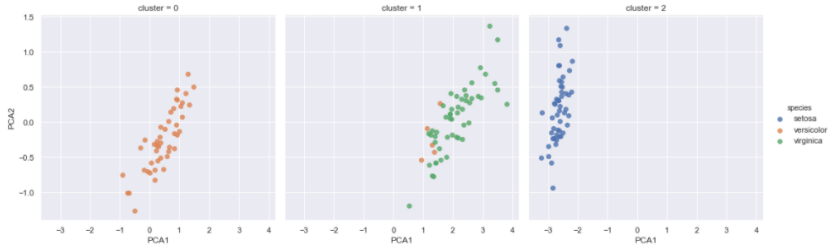
```
In [65]: iris['PCA1'] = X_2D[:, 0]
         iris['PCA2'] = X_2D[:, 1]
         sns.lmplot("PCA1", "PCA2", hue='species', data=iris, fit_reg=False);
```



Unsupervised Iris Custering GMM: Gaussian Mixture Models

In [70]:
```
from sklearn import mixture      # 1. Choose the model class
model = mixture.GaussianMixture(n_components=3,
           covariance_type='full')  # 2. Instantiate the model with hyperparameters
model.fit(X_iris)                   # 3. Fit to data. Notice y is not specified!
y_gmm = model.predict(X_iris)
```

In [71]:
```
iris['cluster'] = y_gmm
sns.lmplot("PCA1", "PCA2", data=iris, hue='species',
           col='cluster', fit_reg=False);
```



Luís Garmendia

GMM: Gaussian Mixture Models

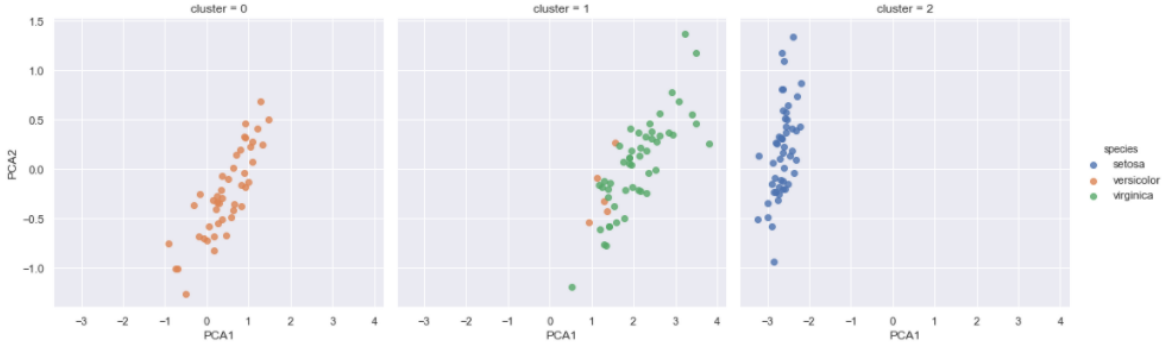
```
from sklearn import mixture
model = mixture.GaussianMixture(n_components=3,
covariance_type='full')
model.fit(X_iris)
y_gmm = model.predict(X_iris)


iris['cluster'] = y_gmm
sns.lmplot(x="PCA1", y="PCA2", data=iris, hue='species', col='cluster',
fit_reg=False);
```

GMM: Gaussian Mixture Models

In [70]:
```
from sklearn import mixture      # 1. Choose the model class
model = mixture.GaussianMixture(n_components=3,
            covariance_type='full')  # 2. Instantiate the model with hyperparameters
model.fit(X_iris)                    # 3. Fit to data. Notice y is not specified!
y_gmm = model.predict(X_iris)
```

In [71]:
```
iris['cluster'] = y_gmm
sns.lmplot("PCA1", "PCA2", data=iris, hue='species',
           col='cluster', fit_reg=False);
```

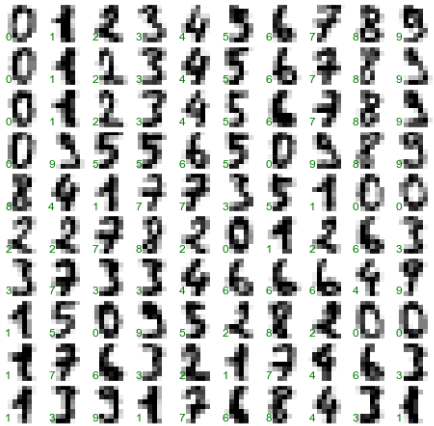


Loading Digits Data
Confusion Matrix

In [73]:
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(10, 10, figsize=(8, 8),
                         subplot_kw={'xticks':[], 'yticks':[]},
                         gridspec_kw=dict(hspace=0.1, wspace=0.1))

for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='binary', interpolation='nearest')
    ax.text(0.05, 0.05, str(digits.target[i]),
            transform=ax.transAxes, color='green')
```



Luís Garmendia

Loading digits data

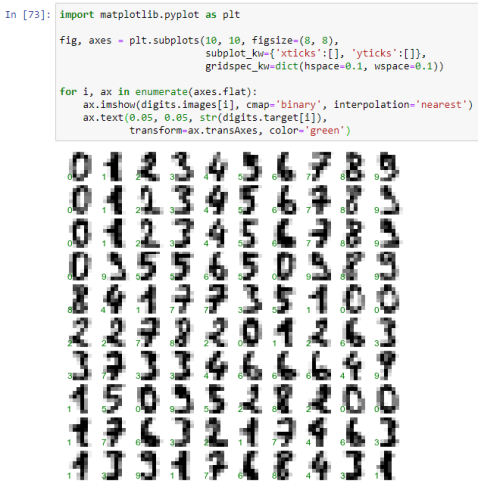
```
from sklearn.datasets import load_digits
digits = load_digits()
digits.images.shape
```

```
In [72]: from sklearn.datasets import load_digits
         digits = load_digits()
         digits.images.shape

Out[72]: (1797, 8, 8)
```

Loading digits data

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(10, 10, figsize=(8, 8),
       subplot_kw={'xticks':[], 'yticks':[]}, gridspec_kw=dict(hspace=0.1, wspace=0.1))
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='binary', interpolation='nearest')
    ax.text(0.05, 0.05, str(digits.target[i]), transform=ax.transAxes, color='green')
```



[n_samples, n_features]

```
X = digits.data
X.shape
y = digits.target
y.shape
```

```
In [74]: X = digits.data
         X.shape

Out[74]: (1797, 64)

In [75]: y = digits.target
         y.shape

Out[75]: (1797,)
```

Isomap Dimensional reduction

```
from sklearn.manifold import Isomap
iso = Isomap(n_components=2)
iso.fit(digits.data)
data_projected = iso.transform(digits.data)
data_projected
```

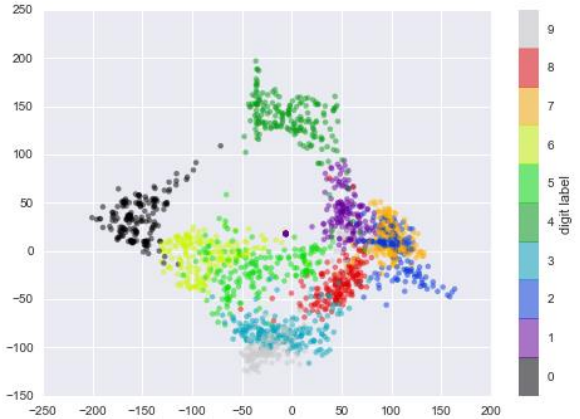
```
In [76]: from sklearn.manifold import Isomap
         iso = Isomap(n_components=2)
         iso.fit(digits.data)
         data_projected = iso.transform(digits.data)
         data_projected.shape

Out[76]: (1797, 2)
```

View Dimensional reduction

```
plt.scatter(data_projected[:, 0], data_projected[:, 1], c=digits.target, edgecolor='none',
alpha=0.5, cmap=plt.cm.get_cmap('rainbow', 10))
plt.colorbar(label='digit label', ticks=range(10))
plt.clim(-0.5, 9.5);
```

```
In [27]: plt.scatter(data_projected[:, 0], data_projected[:, 1], c=digits.target,
             edgecolor='none', alpha=0.5,
             cmap=plt.cm.get_cmap('spectral', 10))
plt.colorbar(label='digit label', ticks=range(10))
plt.clim(-0.5, 9.5);
```



Clasify digits with Naïve Bayes

```
Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, random_state=0)

from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(Xtrain, ytrain)
y_model = model.predict(Xtest)

from sklearn.metrics import accuracy_score
accuracy_score(ytest, y_model)
```

```
In [80]: Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, random_state=0)

In [81]: from sklearn.naive_bayes import GaussianNB
         model = GaussianNB()
         model.fit(Xtrain, ytrain)
         y_model = model.predict(Xtest)

In [82]: from sklearn.metrics import accuracy_score
         accuracy_score(ytest, y_model)

Out[82]: 0.8333333333333334
```

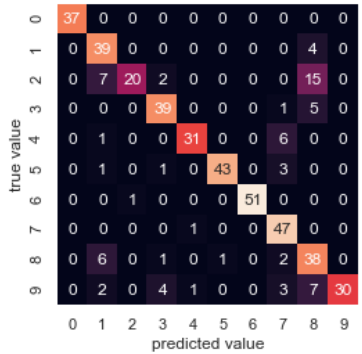
Confusion Matrix

```
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(ytest, y_model)
sns.heatmap(mat, square=True, annot=True, cbar=False)
plt.xlabel('predicted value')
plt.ylabel('true value');
```

```
In [83]: from sklearn.metrics import confusion_matrix

         mat = confusion_matrix(ytest, y_model)

         sns.heatmap(mat, square=True, annot=True, cbar=False)
         plt.xlabel('predicted value')
         plt.ylabel('true value');
```



Confusion Matrix

```
fig, axes = plt.subplots(10, 10, figsize=(8, 8), subplot_kw={'xticks':[], 'yticks':[]},
gridspec_kw=dict(hspace=0.1, wspace=0.1))

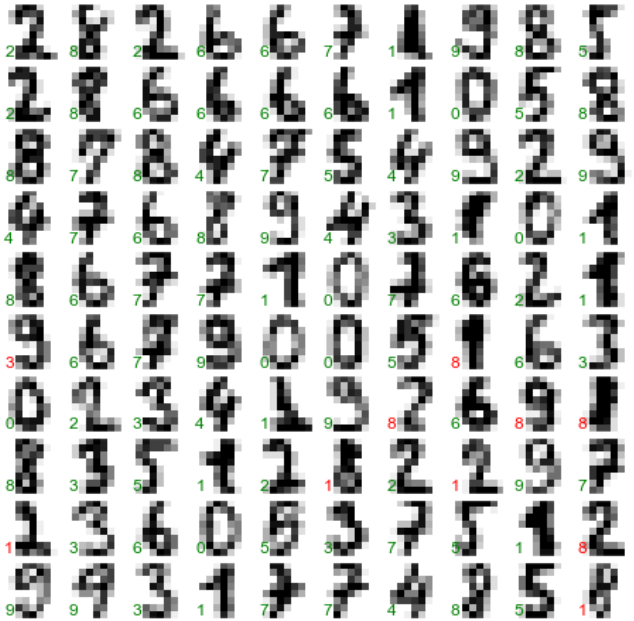
test_images = Xtest.reshape(-1, 8, 8)

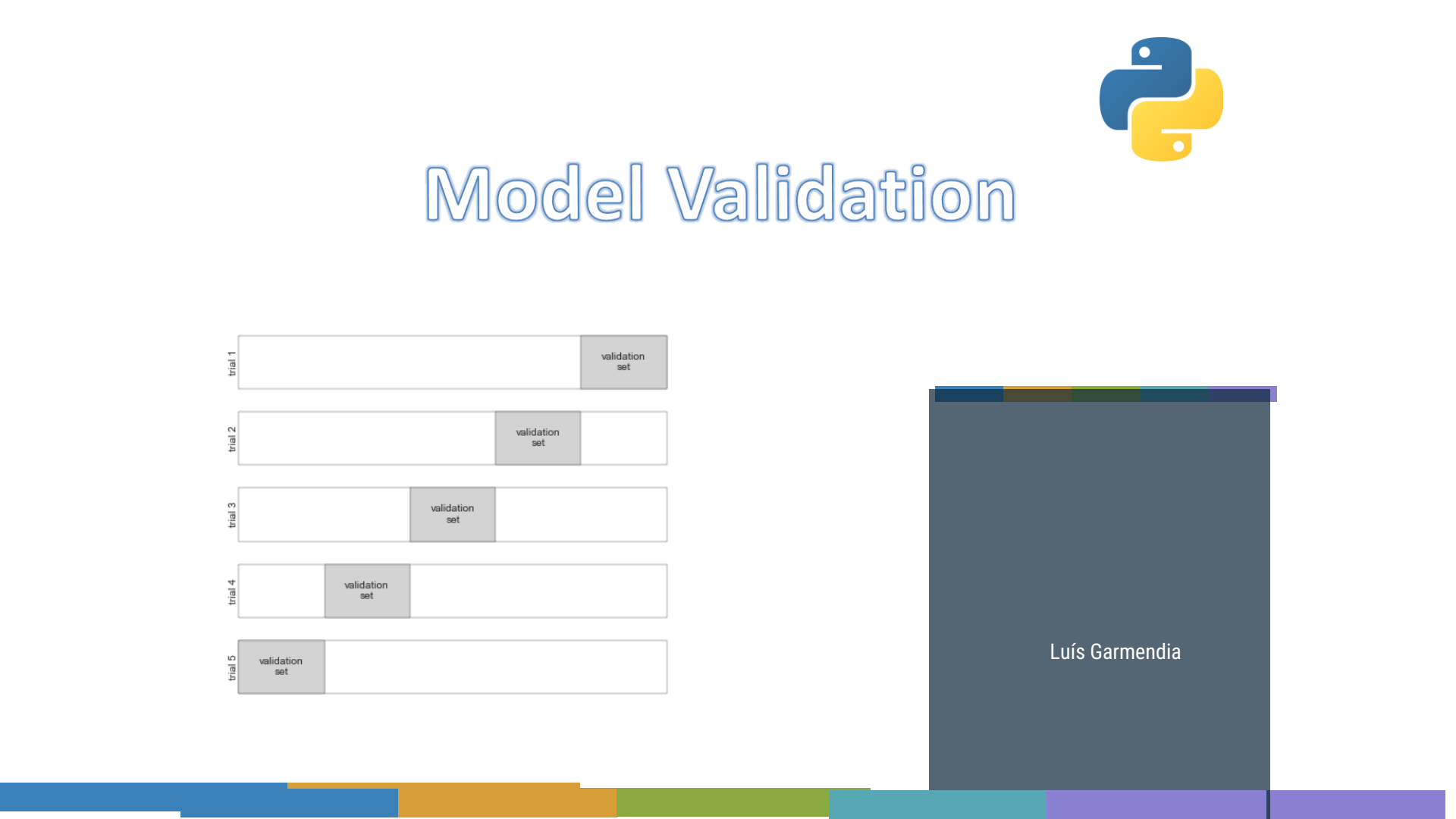
for i, ax in enumerate(axes.flat):
        ax.imshow(test_images[i], cmap='binary', interpolation='nearest')
        ax.text(0.05, 0.05, str(y_model[i]),
                transform=ax.transAxes,
                color='green' if (ytest[i] == y_model[i]) else 'red')
```

In [84]:
```
fig, axes = plt.subplots(10, 10, figsize=(8, 8),
                         subplot_kw={'xticks':[], 'yticks':[]},
                         gridspec_kw=dict(hspace=0.1, wspace=0.1))

test_images = Xtest.reshape(-1, 8, 8)

for i, ax in enumerate(axes.flat):
    ax.imshow(test_images[i], cmap='binary', interpolation='nearest')
    ax.text(0.05, 0.05, str(y_model[i]),
            transform=ax.transAxes,
            color='green' if (ytest[i] == y_model[i]) else 'red')
```





Model Validation

Luís Garmendia

No Validar con datos del entrenamiento

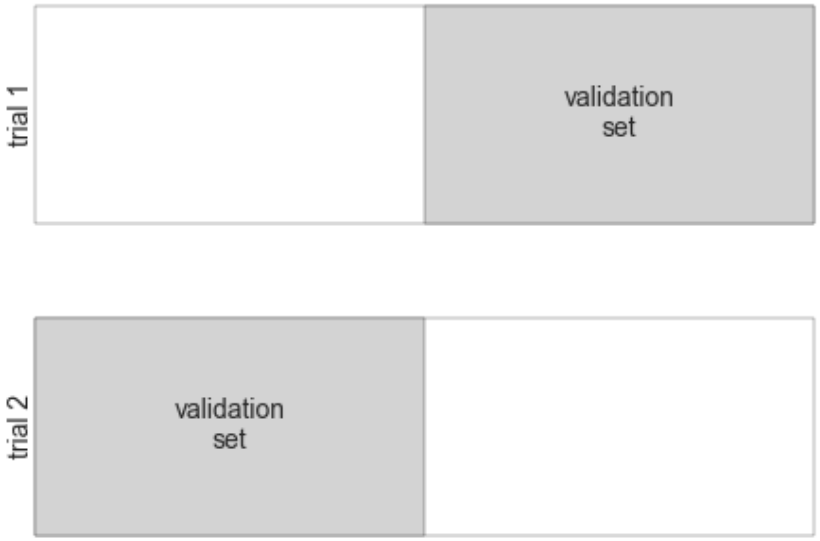
```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
from sklearn.neighbors import
KNeighborsClassifier model =
KNeighborsClassifier(n_neighbors=1)
model.fit(X, y)
y_model = model.predict(X)
from sklearn.metrics import accuracy_score
accuracy_score(y, y_model)
1.0
```

Model_selection.train_test_split

```
from sklearn.model_selection import
train_test_split
X1, X2, y1, y2 = train_test_split(X, y,
random_state=0, train_size=0.5)
model.fit(X1, y1)
y2_model = model.predict(X2)
accuracy_score(y2, y2_model)
```

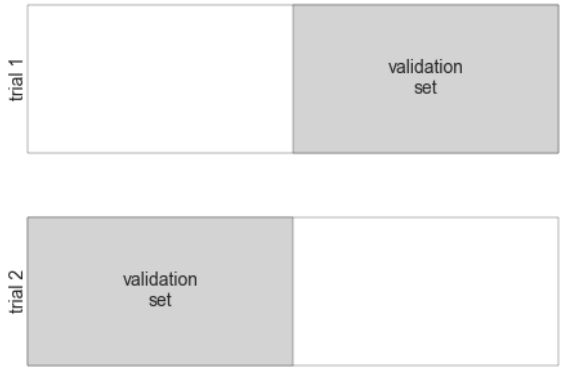
0.9066666666666662

Cross-validation model validation



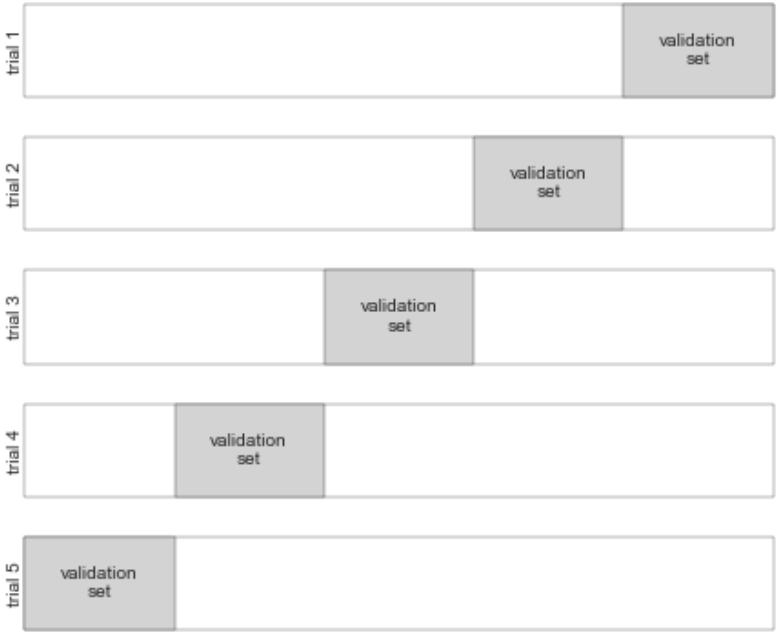
2 validation trials

```
y2_model = model.fit(X1, y1).predict(X2)
y1_model = model.fit(X2, y2).predict(X1)
accuracy_score(y1, y1_model), accuracy_score(y2, y2_model)
(0.9599999999999996, 0.9666666666666662)
```



many validation trials

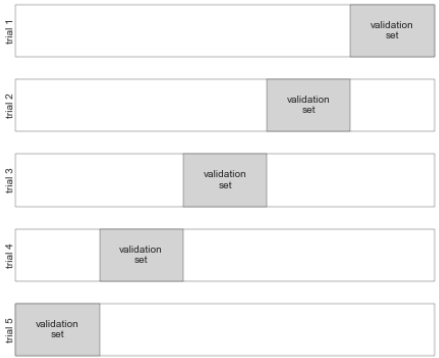
5 cross-validation:



many validation trials
cross_val_score

5 cross-validation:

```
from sklearn.model_selection import cross_val_score
cross_val_score(model, X, y, cv=5)
array([ 0.96666667, 0.96666667, 0.93333333, 0.93333333, 1. ])
```



Leave one out validation trial

```
from sklearn.model_selection import LeaveOneOut
scores = cross_val_score(model, X, y, cv=LeaveOneOut())
Scores
```

```
array([ 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 1., 0., 1., 1., 1., 1., 1.,
1., 1., 1., 1., 1., 0., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 0., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.,
1., 0., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 0., 1., 1., 1., 1.,
1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

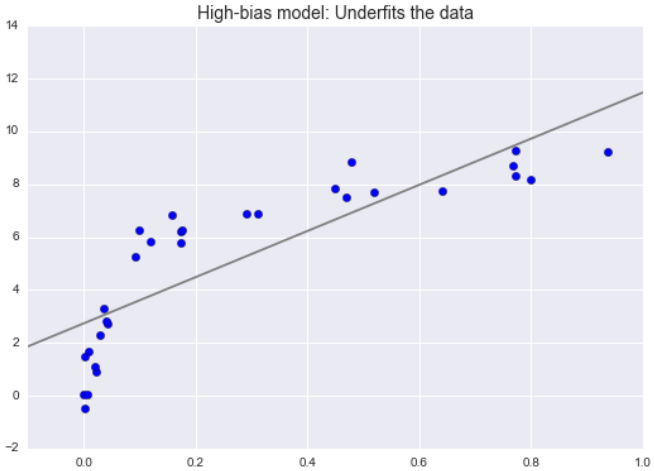
Leave one out validation trial

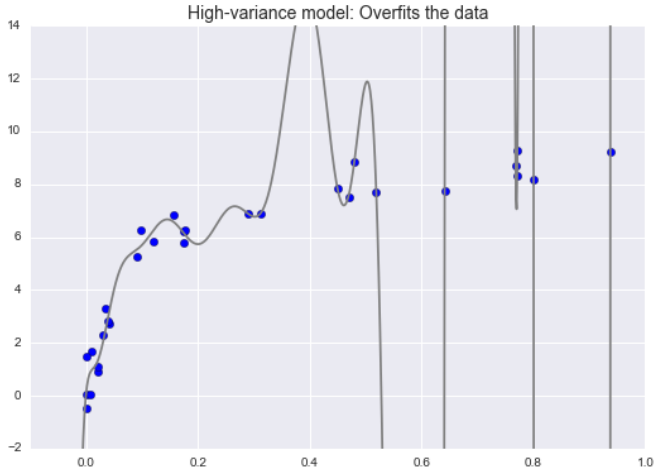
La validación puede hacerse con la media

```
scores.mean()
0.9599999999999996
```

The bias-variance trade-of: underfit y Overfit

Dos regresiones







Feature Engineering
Vectorization

Luís Garmendia

Categórico a numérico

In [111]:
```
data = [
    {'price': 850000, 'rooms': 4, 'neighborhood': 'Queen Anne'},
    {'price': 700000, 'rooms': 3, 'neighborhood': 'Fremont'},
    {'price': 650000, 'rooms': 3, 'neighborhood': 'Wallingford'},
    {'price': 600000, 'rooms': 2, 'neighborhood': 'Fremont'}
]
```

In [112]:
```
from sklearn.feature_extraction import DictVectorizer
vec = DictVectorizer(sparse=False, dtype=int)
vec.fit_transform(data)
```

Out[112]:
```
array([[     0,      1,      0, 850000,      4],
       [     1,      0,      0, 700000,      3],
       [     0,      0,      1, 650000,      3],
       [     1,      0,      0, 600000,      2]])
```

Categórico a numérico

```
data = [
        {'price': 850000, 'rooms': 4, 'neighborhood': 'Queen Anne'},
        {'price': 700000, 'rooms': 3, 'neighborhood': 'Fremont'},
        {'price': 650000, 'rooms': 3, 'neighborhood': 'Wallingford'},
        {'price': 600000, 'rooms': 2, 'neighborhood': 'Fremont'} ]

from sklearn.feature_extraction import DictVectorizer
vec = DictVectorizer(sparse=False, dtype=int)
vec.fit_transform(data)
```

Categorical Features

Nuevas columnas

vec.get_feature_names_out()

```
In [113]: vec.get_feature_names()

Out[113]: ['neighborhood=Fremont',
           'neighborhood=Queen Anne',
           'neighborhood=Wallingford',
           'price',
           'rooms']
```

Vectorización de textos
word count

```
sample = ['problem of evil', 'evil queen', 'horizon problem']

from sklearn.feature_extraction.text import CountVectorizer
vec = CountVectorizer()
X = vec.fit_transform(sample)
X
```

Vectorización de textos word count

import pandas as pd

pd.DataFrame(X.toarray(), columns=vec.get_feature_names())

```
In [114]: sample = ['problem of evil',
                    'evil queen',
                    'horizon problem']
```

```
In [115]: from sklearn.feature_extraction.text import CountVectorizer

          vec = CountVectorizer()
          X = vec.fit_transform(sample)
          X
```

```
Out[115]: <3x5 sparse matrix of type '<class 'numpy.int64'>'
                  with 7 stored elements in Compressed Sparse Row format>
```

```
In [116]: import pandas as pd
          pd.DataFrame(X.toarray(), columns=vec.get_feature_names())
```

Out[116]:

	evil	horizon	of	problem	queen
0	1	0	1	1	0
1	1	0	0	0	1
2	0	1	0	1	0

Vectorización de textos
TF–IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer()
X = vec.fit_transform(sample)
pd.DataFrame(X.toarray(), columns=vec.get_feature_names())
```

In [117]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer()
X = vec.fit_transform(sample)
pd.DataFrame(X.toarray(), columns=vec.get_feature_names())
```

Out[117]:

	evil	horizon	of	problem	queen
0	0.517856	0.000000	0.680919	0.517856	0.000000
1	0.605349	0.000000	0.000000	0.000000	0.795961
2	0.000000	0.795961	0.000000	0.605349	0.000000

Imputación de nan

```
from numpy import nan
X = np.array([[ nan, 0, 3 ],
              [ 3, 7, 9 ],
              [ 3, 5, 2 ],
              [ 4, nan, 6 ],
              [ 8, 8, 1 ]])
y = np.array([14, 16, -1, 8, -5])
```

Imputación de nan

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='mean')
X2 = imp.fit_transform(X)
X2
```

In [127]:
```
from numpy import nan
X = np.array([[ nan, 0,   3  ],
              [ 3,   7,   9  ],
              [ 3,   5,   2  ],
              [ 4,   nan, 6  ],
              [ 8,   8,   1  ]])
y = np.array([14, 16, -1,  8, -5])
```

In [128]:
```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='mean')
X2 = imp.fit_transform(X)
X2
```

Out[128]:
```
array([[4.5, 0. , 3. ],
       [3. , 7. , 9. ],
       [3. , 5. , 2. ],
       [4. , 5. , 6. ],
       [8. , 8. , 1. ]])
```

Feature pipelines

It can quickly become tedious to do the transformations by hand, especially if you wish to string together multiple steps.

For example, we might want a processing pipeline that looks something like this:

1.Impute missing values using the mean

2.Transform features to quadratic

3.Fit a linear regression

To streamline this type of processing pipeline, Scikit-Learn provides a Pipeline object

Feature pipelines

```
from sklearn.pipeline import make_pipeline
model = make_pipeline(SimpleImputer(strategy='mean'),
        PolynomialFeatures(degree=2),
        LinearRegression())
```

1.Impute missing values using the mean

2.Transform features to quadratic

3.Fit a linear regression

```
model.fit(X, y) # X with missing values print(y)
print(model.predict(X))
```

Referencias

https://ipython.org/

https://numpy.org/

https://jupyter.org/try

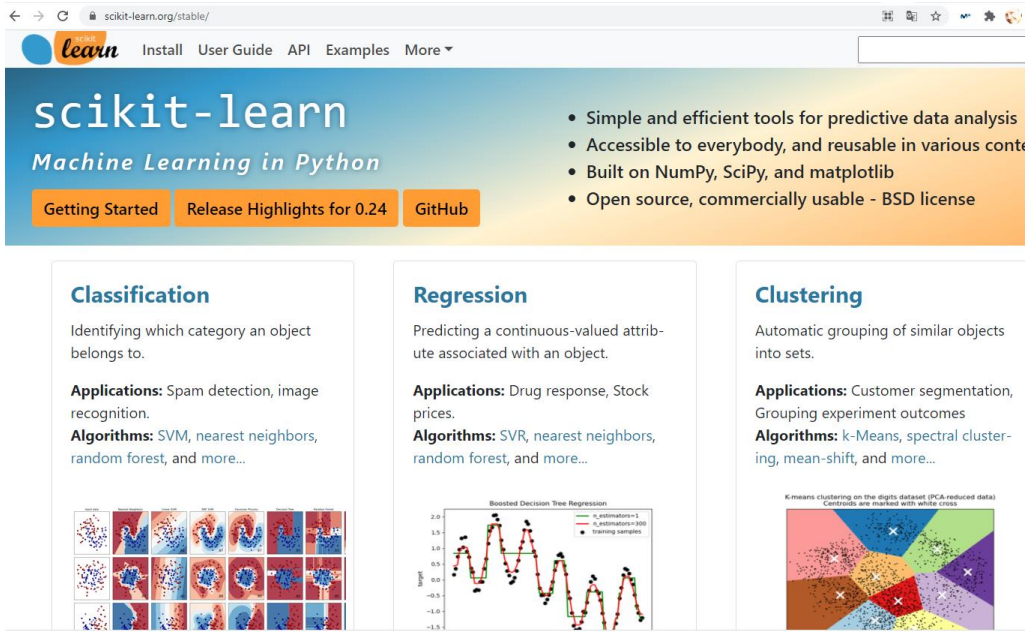
https://pandas.pydata.org/

http://seaborn.pydata.org/

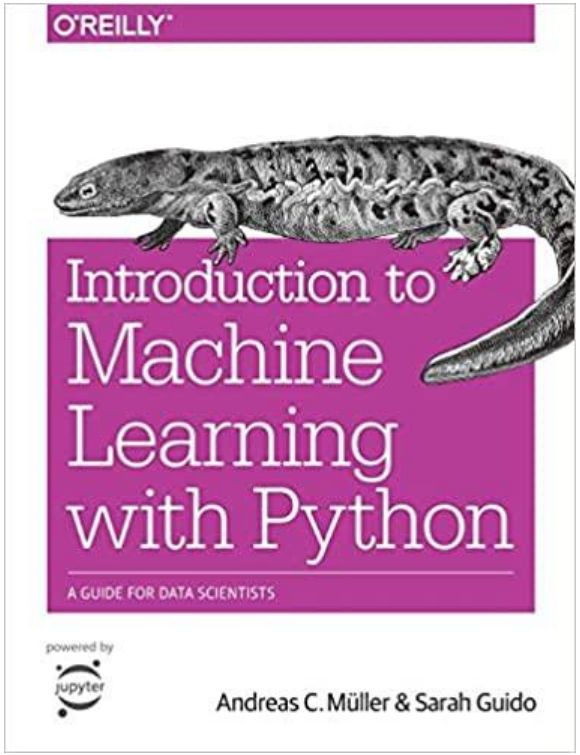
https://scikit-learn.org/stable/

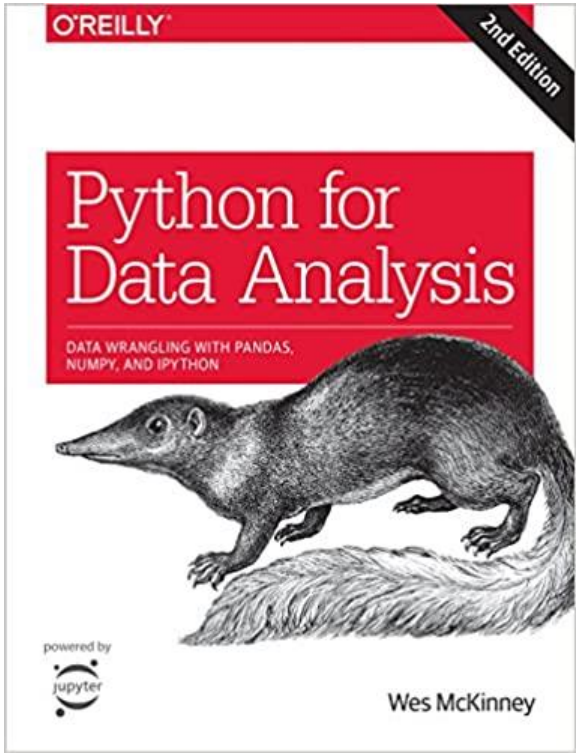
Referencias

https://scikit-learn.org/stable/

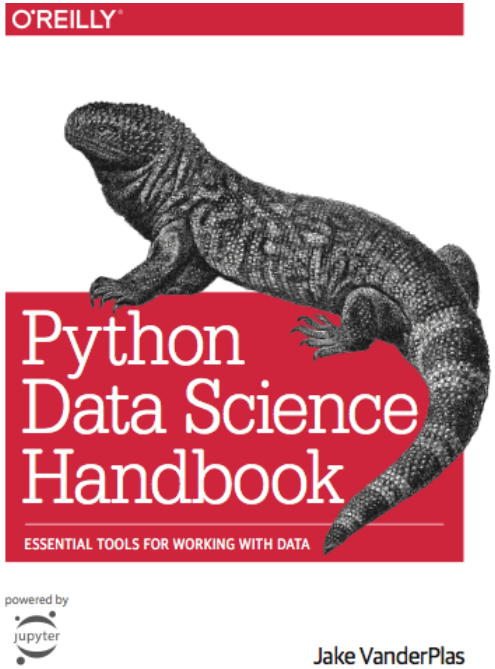


Referencias





Referencias



https://jakevdp.github.io/PythonDataScienceHandbook/

ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE DE MADRID

